Artificial Intelligence:
Knowledge Representation and Reasoning

The Resolution Refutation Method

Deepak Khemani

Department of Computer Science & Engineering

IIT Madras

Interpretation 1

{(O A B), (O B C), (not (M A)), (M C)}

Domain: Blocks World

Predicate symbols

O → On

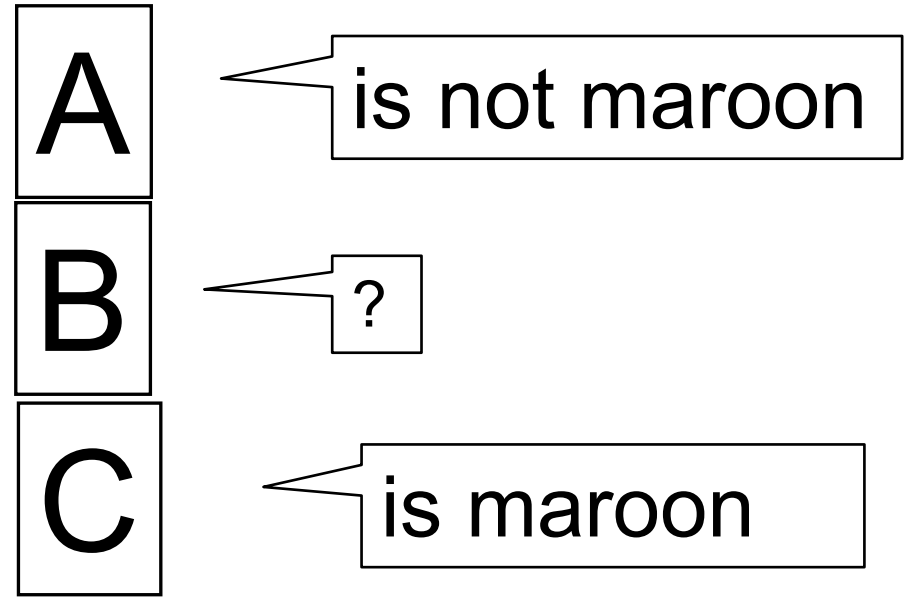
M → Maroon

Constant Symbols

A, B, C → blocks

A is on B

B is on C



Recap

Interpretation 2

{(O A B), (O B C), (not (M A)), (M C)}

Domain: People

Predicate symbols
O → LookingAt
M → Married

Constant Symbols
A → Archie
B → Betty
C → Cuthbert

Betty is looking at Cuthbert

Archie is looking at Betty

Recap

The Goal

{(O A B), (O B C), (not (M A)), (M C)}

Given the KB and the goal

(exists (x y) (and (O x y) (not (M x)) (M y)))

or equivalently (and (O ?x ?y) (not (M ?x)) (M ?y))

…is clearly entailed

Interpretations are,

Blocks World: *Is there a not-maroon block on a maroon block?*

People: *Is a not-married person looking at a married one?*

Recap

Incompleteness of Backward and Forward Chaining

Given the KB,

{(O A B), (O B C), (not (M A)), (M C)}

And the Goal,

(and (O ?x ?y) (not (M ?x)) (M ?y))

Neither Forward Chaining nor Backward Chaining is able to generate a proof.

Both are Incomplete!

Next, we look at a proof method,
the Resolution Refutation System,
that is Sound and Complete for FOL

Recap

Resolution Method in Propositional Logic

The resolution method requires that the formula is in *conjunctive normal form* (*CNF*). A formula *F* is in *CNF* if it has the following structure.

$$F = C_1 \wedge C_2 \wedge \ldots \wedge C_n$$

That is, the formula is a conjunction of clauses where each clause C_i is a disjunction of literals,

$$C_i = D_{i1} \vee D_{i2} \vee \ldots \vee D_{ik}$$

Each literal is either a proposition or the negation of a proposition.

$$D_i = L \text{ or } \neg L$$

A formula in *CNF* is also represented as a set of sets.

$$F = \{\{D_{11}, D_{12}, \ldots, D_{1k(1)}\}, \ldots, \{D_{n1}, D_{n2}, \ldots, D_{nk(n)}\}\}$$

Any formula → CNF

Any formula in propositional logic may be converted into *CNF* by the use of substitution rules based on the tautological equivalences. Conversion into *CNF* generally results in an *increase* in the size of the formula, often reaching *exponential number of clauses* in the number of propositions.

$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma))$	associativity of $\vee$
$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma))$	associativity of $\wedge$
$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta)$	DeMorgan's Law
$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta)$	DeMorgan's Law
$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma))$	distributivity of $\wedge$ over $\vee$
$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma))$	distributivity of $\vee$ over $\wedge$
$(\alpha \supset \beta) \equiv (\neg\beta \supset \neg\alpha)$	contrapositive
$(\alpha \supset \beta) \equiv (\neg\alpha \vee \beta)$	implication

The Resolution Rule

The single rule used in the refutation method, called the resolution rule, takes two clauses that have a complimentary literal as follows.

From: $R_1 \vee R_2 \ldots \vee R_k \vee Q$

And: $P_1 \vee P_2 \ldots \vee P_m \vee \neg Q$

Infer: $R_1 \vee R_2 \ldots \vee R_k \vee P_1 \vee P_2 \ldots \vee P_m$

Both the literal Q and its negation $\neg Q$ are removed, and the remaining literals are combined to form a new clause, called the *resolvent*. With the smaller clause the rule becomes,

From: $R \vee Q$

And: $P \vee \neg Q$

Infer: $R \vee P$

Validity of the Resolution Rule

The validity of the resolution rule can be established by showing that adding the resolvent does not change the set of clauses logically. That is, the sets before and after are equivalent. It suffices to show that,

$$((R \vee Q) \wedge (P \vee \neg Q)) \equiv ((R \vee Q) \wedge (P \vee \neg Q) \wedge (R \vee P))$$

Deduction Theorem

Given a set of premises $\{\alpha_1, \alpha_2, \ldots, \alpha_N\}$ and the desired goal β, we want to determine if the formula,

$$((\alpha_1 \wedge \alpha_2 \wedge \ldots \wedge \alpha_n) \supset \beta)$$

is true. This follows from the well known Deduction theorem that asserts that,

$$\{\alpha_1, \alpha_2 \ldots, \alpha_n\} \models \beta \quad \text{iff} \quad \models ((\alpha_1 \wedge \alpha_2 \wedge \ldots \wedge \alpha_n) \supset \beta)$$

Also $((\alpha_1 \wedge \alpha_2 \wedge \ldots \wedge \alpha_n) \supset \beta)$ is a *tautology*

iff $\neg((\alpha_1 \wedge \alpha_2 \wedge \ldots \wedge \alpha_n) \supset \beta)$ is *unsatisfiable*

Proof by Contradiction

To show that $\neg((\alpha_1 \wedge \alpha_2 \wedge \ldots \wedge \alpha_n) \supset \beta)$ is unsatisfiable, one can add the negation of the goal to the set of premises.

$\neg((\alpha_1 \wedge \alpha_2 \wedge \ldots \wedge \alpha_n) \supset \beta) \equiv \neg(\neg(\alpha_1 \wedge \alpha_2 \wedge \ldots \wedge \alpha_n) \vee \beta)$

$\equiv ((\alpha_1 \wedge \alpha_2 \wedge \ldots \wedge \alpha_n) \wedge \neg\beta)$

If the resulting set of formulas $((\alpha_1 \wedge \alpha_2 \wedge \ldots \wedge \alpha_n) \wedge \neg\beta)$ is unsatisfiable so is the original formula $\neg((\alpha_1 \wedge \alpha_2 \wedge \ldots \wedge \alpha_n) \supset \beta)$.

Then $((\alpha_1 \wedge \alpha_2 \wedge \ldots \wedge \alpha_n) \supset \beta)$ is a tautology

Resolution Refutation

Show that $\neg((\alpha_1 \wedge \alpha_2 \wedge \ldots \wedge \alpha_n) \supset \beta)$ is unsatisfiable. We convert this into *CNF*, the form that is required by the resolution method.

$$\neg((\alpha_1 \wedge \alpha_2 \wedge \ldots \wedge \alpha_n) \supset \beta) \equiv \neg(\neg(\alpha_1 \wedge \alpha_2 \wedge \ldots \wedge \alpha_n) \vee \beta)$$

$$\equiv ((\alpha_1 \wedge \alpha_2 \wedge \ldots \wedge \alpha_n) \wedge \neg\beta)$$

$$\equiv (\alpha'_1 \wedge \alpha'_2 \wedge \ldots \wedge \alpha'_n \wedge \neg\beta')$$

To generate the input for the resolution method we simply need to negate the goal and add it to the set of clauses.

We may need to convert each of the premises and the negated goal into the clause (*CNF*) form denoted by α'_i and β' above.

The basic algorithm

Given a set of clauses, pick any two clauses and add the resolvent to the set.

If at any time we generate a resolvent that is the empty clause (or null clause) then the procedure can terminate.

This is because the empty clause, or $\perp$, evaluates to false. We also use the symbol $\square$ to stand for the empty or null clause.

An empty clause can be generated from two clauses S and $\neg S$ by the application of the resolution rule.

Now if a conjunctive formula (the database) has both S and $\neg S$ then it must be false

To show that a formula is unsatisfiable

Let the original formula be $\{P_1, P_2, \ldots, P_N\}$. Remember this stands for a conjunction of the N clauses. To this we add a sequence of resolvents $R_1, R_2, R_3, \ldots$ culminating with $\perp$. The databases at all stages are logically equivalent, because the resolution rule is sound.

$$\{P_1, P_2, \ldots, P_N\} \equiv \{P_1, P_2, \ldots, P_N, R_1\}$$
$$\equiv \{P_1, P_2, \ldots, P_N, R_1, R_2\}$$
$$\equiv \{P_1, P_2, \ldots, P_N, R_1, R_2, R_3\}$$
$$\equiv \{P_1, P_2, \ldots, P_N, R_1, R_2, R_3, \ldots, \perp\}$$

Now since the last set of clauses evaluates to *false* (because it contains the empty clause) the set we started with, which is logically equivalent, also evaluates to *false*.

Thus $\{P_1, P_2, \ldots, P_N\}$ is *unsatisfiable*.

The Resolution Refutation Method

The Resolution Refutation method was devised by Alan Robinson in 1965. He showed that the Resolution Refutation method is complete for deriving the null clause.

A proof by resolution method is a *proof by contradiction*. We start with the set of premises, *negate* the goal and add it as another clause, and show that it *leads to a contradiction* (something that is false or not possible).

Completeness: If the input formula is unsatisfiable there always exists a derivation of the null clause.

The Alice problem revisited

Alice likes mathematics and she likes stories. If she likes mathematics she likes algebra. If she likes algebra and likes physics she will go to college. She does not like stories or she likes physics. She does not like chemistry and history.

Encoding: P = Alice likes mathematics. Q = Alice likes stories. R = Alice likes algebra. S = Alice likes physics. T = Alice will go to college. U = Alice likes chemistry. V = Alice likes history.

Then the given facts are,

$(P \wedge Q)$

$(P \supset R)$

$((R \wedge S) \supset T)$

$(\sim Q \vee S)$

$(\sim U \wedge \sim V)$

Will Alice go to college? That is, Is T *true*?

Formulating the problem

To show that the following formula is a tautology.

$(((P \wedge Q) \wedge (P \supset R) \wedge ((R \wedge S) \supset T) \wedge (\neg Q \vee S) \wedge (\neg U \wedge \neg V)) \supset T)$

- We take each of the premises and convert it to a clause.
- The first premise ($P \wedge Q$) gives us two clauses, as does the last one.
- We also add the negation of the goal $\neg T$ as a clause.
- Then we apply the resolution rule repeatedly

  -- till the null clause is derived.

It is known the method is decidable for PL

Deriving the null clause

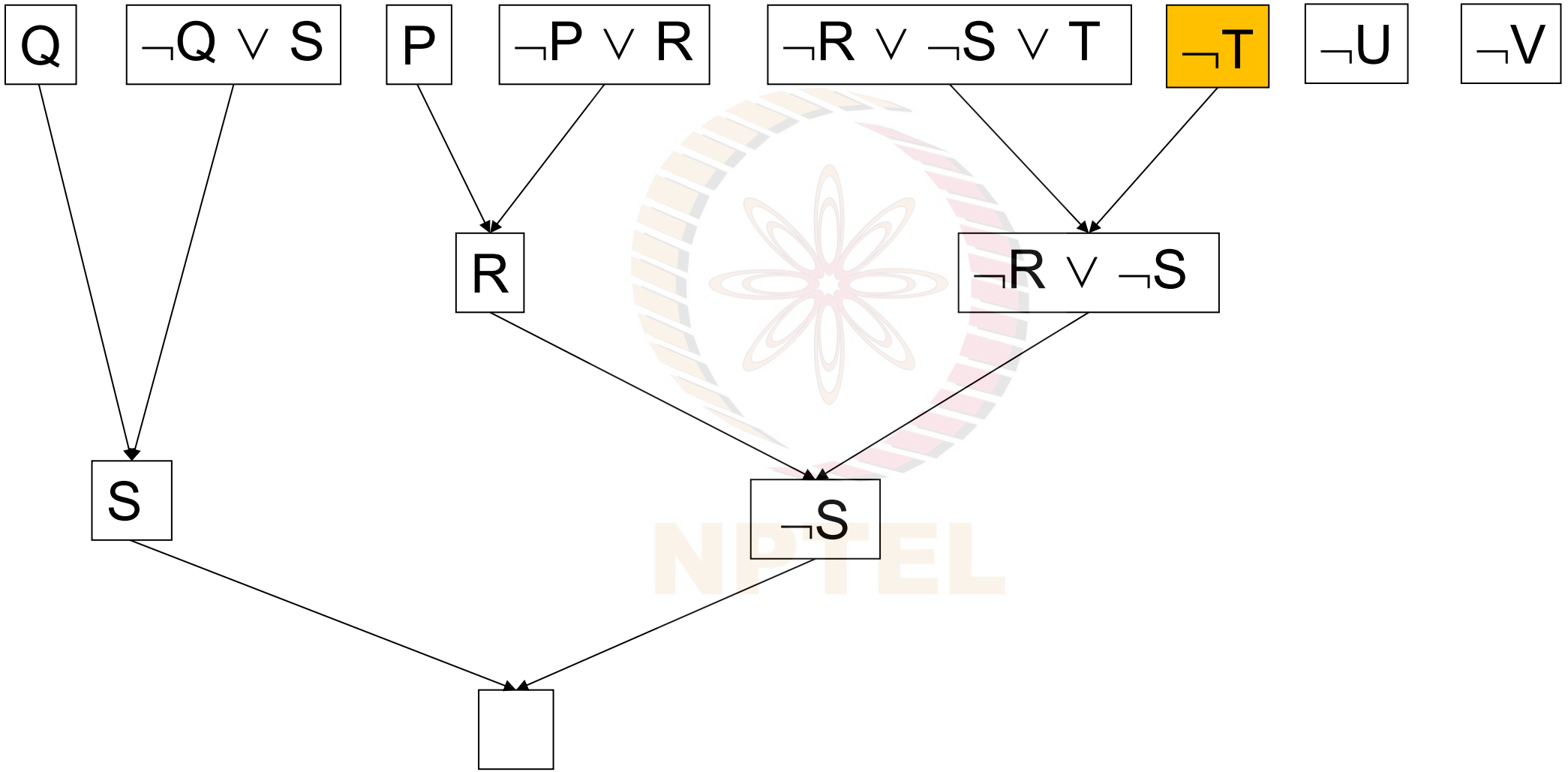
1. P
2. Q
3. ¬P ∨ R
4. ¬R ∨ ¬S ∨ T
5. ¬Q ∨ S
6. ¬U
7. ¬V
8. ¬T — negated goal

The resolvents are,

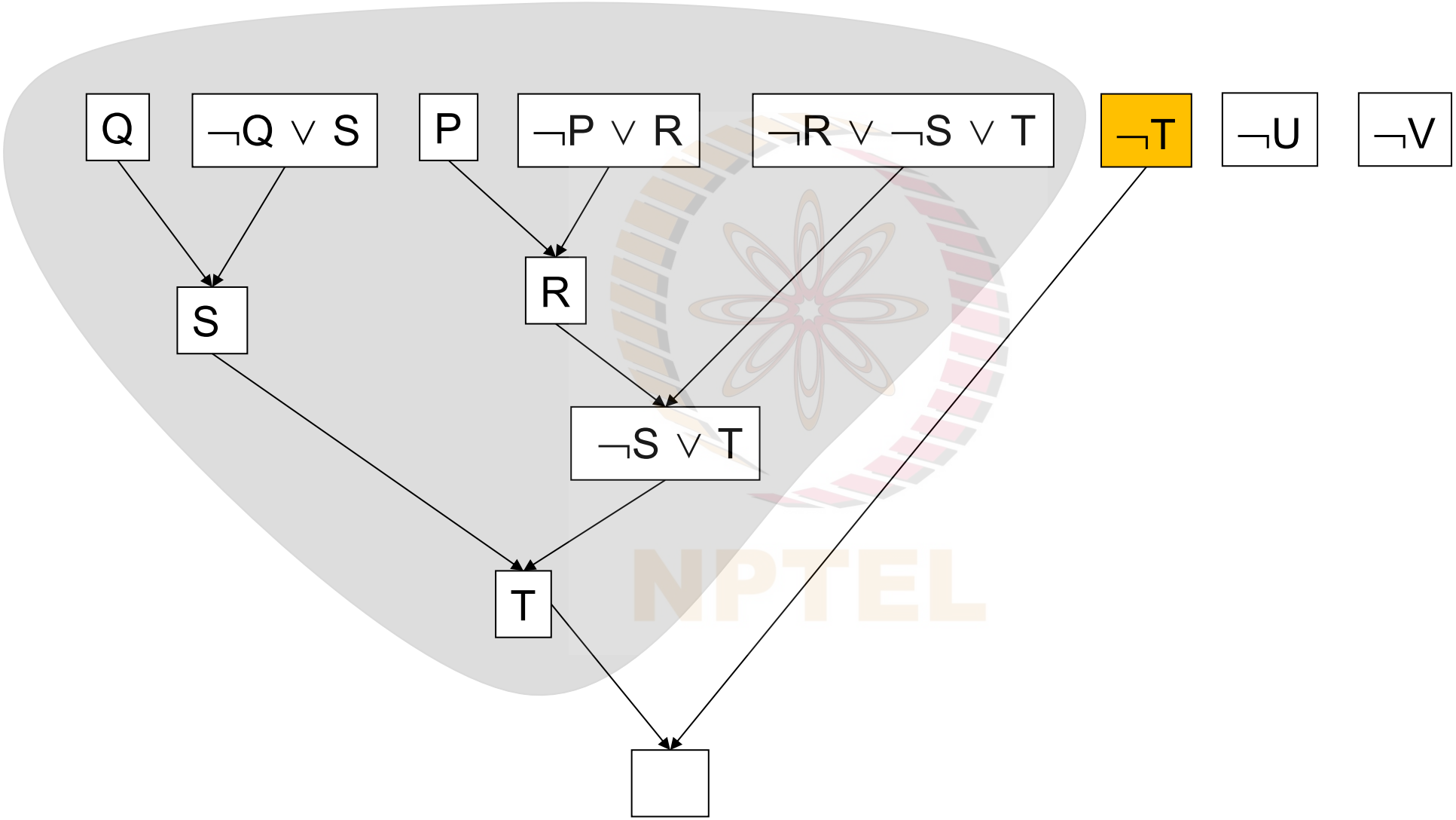
9. ¬R ∨ ¬S — from 4, 8
10. R — from 1, 3
11. ¬S — from 9, 10
12. ¬Q — from 11, 5
13. □ — from 2, 12

q.e.d

The Proof as a Directed Acyclic Graph (DAG)



An alternative derivation



Clause form in FOL

A first order formula is in clause form if it is of the following form,

$$\forall x_1 \forall x_2 \ldots \forall x_V (C_1 \wedge C_2 \ldots \wedge C_N)$$

where each C_i is a *clause* made of disjunction of *literals*, and each literal is an atomic formula or its negation

The clause form contains only universal quantifiers.

The consequence of having only universal quantifiers and all of them bunched up in the left is that one can in fact ignore the quantifiers during processing.

That makes writing programs a little bit simpler.

That is, we work with the Implicit Quantifier form.

Converting to Clause Form

It was shown by Thoralf Skolem that every formula can be converted into the clause form. Take the existential closure of α. This ensures that there are no free variables in the formula.

1. Standardize variables apart across quantifiers. Rename variables so that the same symbol does not occur in different quantifiers.
2. Eliminate all occurrences of operators other than $\wedge$, $\vee$, and $\neg$.
3. Move $\neg$ all the way in.
4. Push the quantifiers to the right. This ensures that their scope is as tight a possible.
5. Eliminate $\exists$.
6. Move all $\forall$ to the left. They can be ignored henceforth.
7. Distribute $\wedge$ over $\vee$.
8. Simplify
9. Rename variables in each clause (disjunction).

Conversion: an example

$(girl(y) \wedge \forall x\ (boy(x) \supset likes(x, y)) \vee \exists x \exists z\ (boy(x) \wedge girl(z) \wedge loves(x,z))$

1. $\exists y((girl(y) \wedge \forall x\ (boy(x) \supset likes(x, y)) \vee \exists x \exists z(boy(x) \wedge girl(z) \wedge\ loves(x,z)))$

Standardize variables apart

2. $\exists y((girl(y) \wedge \forall x(boy(x) \supset likes(x,y)) \vee \exists x_1 \exists z(boy(x_1) \wedge girl(z) \wedge loves(x_1,z)))$

Eliminate $\supset$

3. $\exists y((girl(y) \wedge \forall x(\neg boy(x) \vee likes(x,y)) \vee \exists x_1 \exists z(boy(x_1) \wedge girl(z) \wedge loves(x_1,z)))$

4. no change

5. no change

6. $(girl(sk\text{-}y) \wedge \forall x(\neg boy(x) \vee likes(x,sk\text{-}y))) \vee$
   $(boy(sk\text{-}x_1) \wedge girl(sk\text{-}z) \wedge loves(sk\text{-}x_1,sk\text{-}z)))$

Move $\forall x$ to the left.

(girl(y)∧∀x (boy(x)⊃likes(x, y))∨∃x∃z (boy(x)∧girl(z)∧loves(x,z))

7.(∀x((girl(sk-y)∧(¬boy(x)∨likes(x,sk-y))∨(boy(sk-x_1)
∧girl(sk-z)∧loves(sk-x_1,sk-z))) distribute

8. (girl(sk-y)∨boy(sk-x_1)) ∧ (girl(sk-y)∨girl(sk-z)) ∧ (girl(sk-y)∨loves(sk-x_1,sk-z))
∧ (¬boy(x)∨likes(x,sk-y)∨(boy(sk-x_1)) ∧ (¬boy(x)∨likes(x,sk-y)∨girl(sk-z))
∧ (¬boy(x)∨likes(x,sk-y)∨loves(sk-x_1,sk-z))
separate the clauses and standardize variables apart

9. no change

10. (girl(sk-y)∨boy(sk-x_1)) ∧
(girl(sk-y)∨girl(sk-z)) ∧
(girl(sk-y)∨loves(sk-x_1,sk-z)) ∧
(¬boy(x)∨likes(x,sk-y)∨(boy(sk-x_1)) ∧
(¬boy(x_5)∨likes(x_5,sk-y)∨girl(sk-z)) ∧
(¬boy(x_6)∨likes(x_6,sk-y)∨loves(sk-x_1,sk-z))

Resolution Refutation for FOL

As with the propositional logic the procedure for finding a proof by the resolution refutation method is as follows,

1. Convert each premise into clause form
2. Negate the goal and convert it into clause form
3. Add the negated goal to the set of clauses
4. Choose two clauses such that two opposite signed literals in them can be unified
5. Resolve the two clauses using the MGU and add the resolvent to the set
6. Repeat steps 4-5 till a null resolvent is produced

The Resolution Rule for FOL

For the sake of completeness the resolution rule is defined as follows. A literal is an atomic formula. A clause is a disjunction of literals. Let C_i and C_k be two clauses with the structure,

$$C_i = (L_1 \vee L_2 \vee \ldots \vee L_k \vee P_1 \vee P_2 \vee \ldots \vee P_n)$$

$$C_k = (\neg R_1 \vee \neg R_2 \vee \ldots \vee \neg R_s \vee Q_1 \vee Q_2 \vee \ldots \vee Q_t)$$

If θ is the MGU for $\{ L_1, L_2, \ldots , L_k, R_1, R_2, \ldots, R_s\}$ then we can resolve C_i and C_k to give us the resolvent,

$$(P_1\theta \vee P_2\theta \vee \ldots \vee P_n\theta \vee Q_1\theta \vee Q_2\theta \vee \ldots \vee Q_t\theta)$$

That is we throw away all the positive literals L_j
and negative literals $\neg R_i$,
and combine the remainder
after applying the substitution θ.

The Socratic argument

The three clauses for the Socratic argument are,

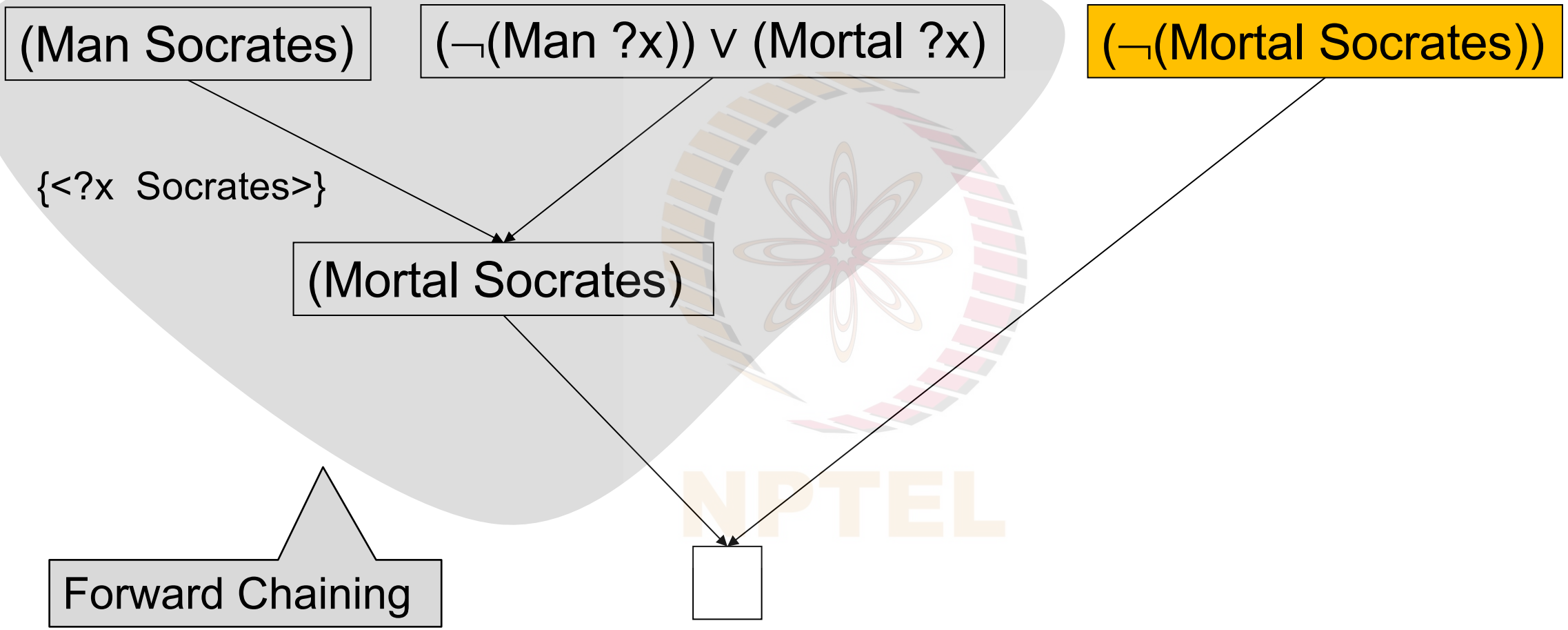
C_1	=	$(\neg(\text{Man ?x})) \vee (\text{Mortal ?x})$	premise
C_2	=	(Man Socrates)	premise
C_3	=	$(\neg(\text{Mortal Socrates}))$	negated goal

The following resolvents are generated,

R_1	=	(Mortal Socrates)	C_1, C_2, {<?x Socrates>}
R_2	=	$\perp$	C_3, R_1

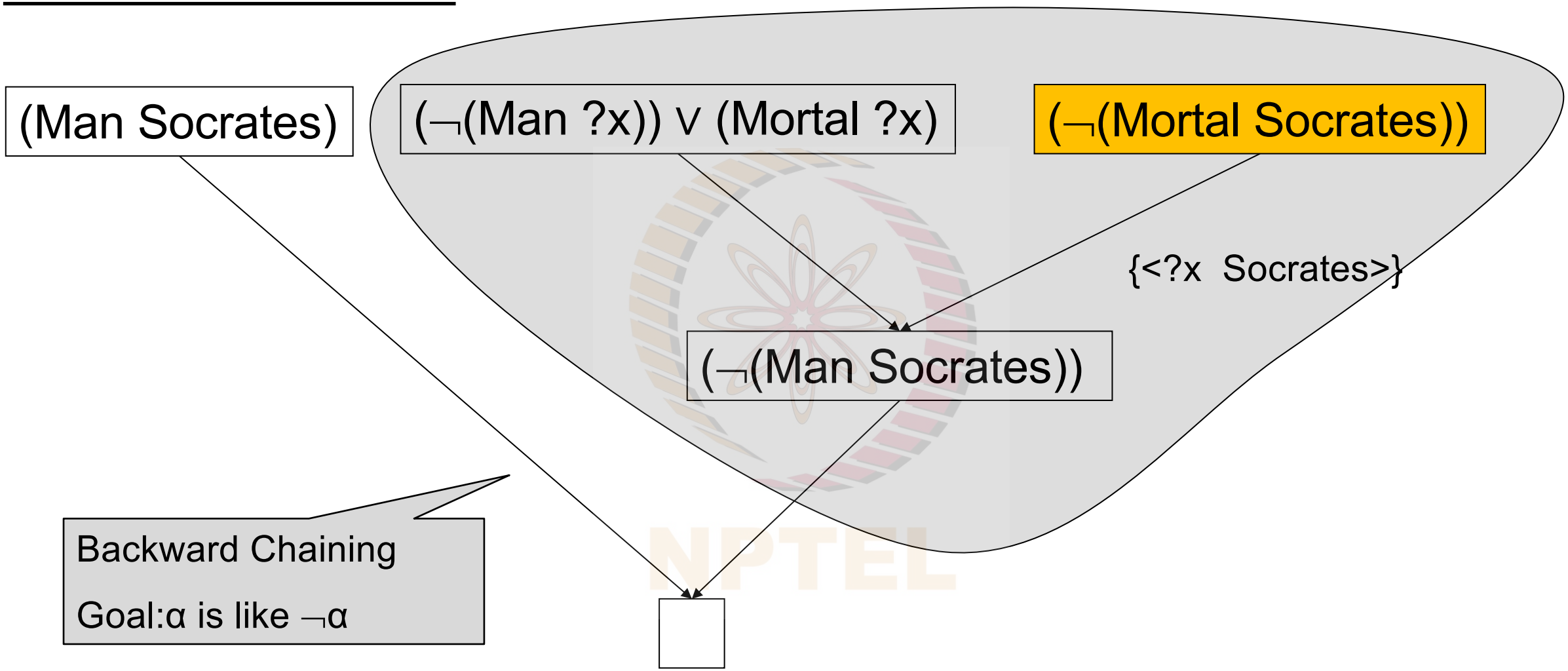
Remember that applying a substitution {?x = socrates}
is a kind of instantiation.

The Proof as a Directed Acyclic Graph (DAG)



The Resolution Rule subsumes Forward Chaining

Another derivation



The Resolution Rule subsumes Backward Chaining

The Alice problem

The clauses are

A curiously mixed notation

1.(likes Alice Math)

2.(likes Alice stories)

3.(¬(likes ?x Math)) ∨ (likes ?x Algebra)

4.(¬(likes ?y Algebra)) ∨ (¬(likes ?y Physics)) ∨ (goesTo ?y College)

5.(¬(likes Alice stories)) ∨ (likes Alice Physics)

6.(¬(likes Alice Chemistry))

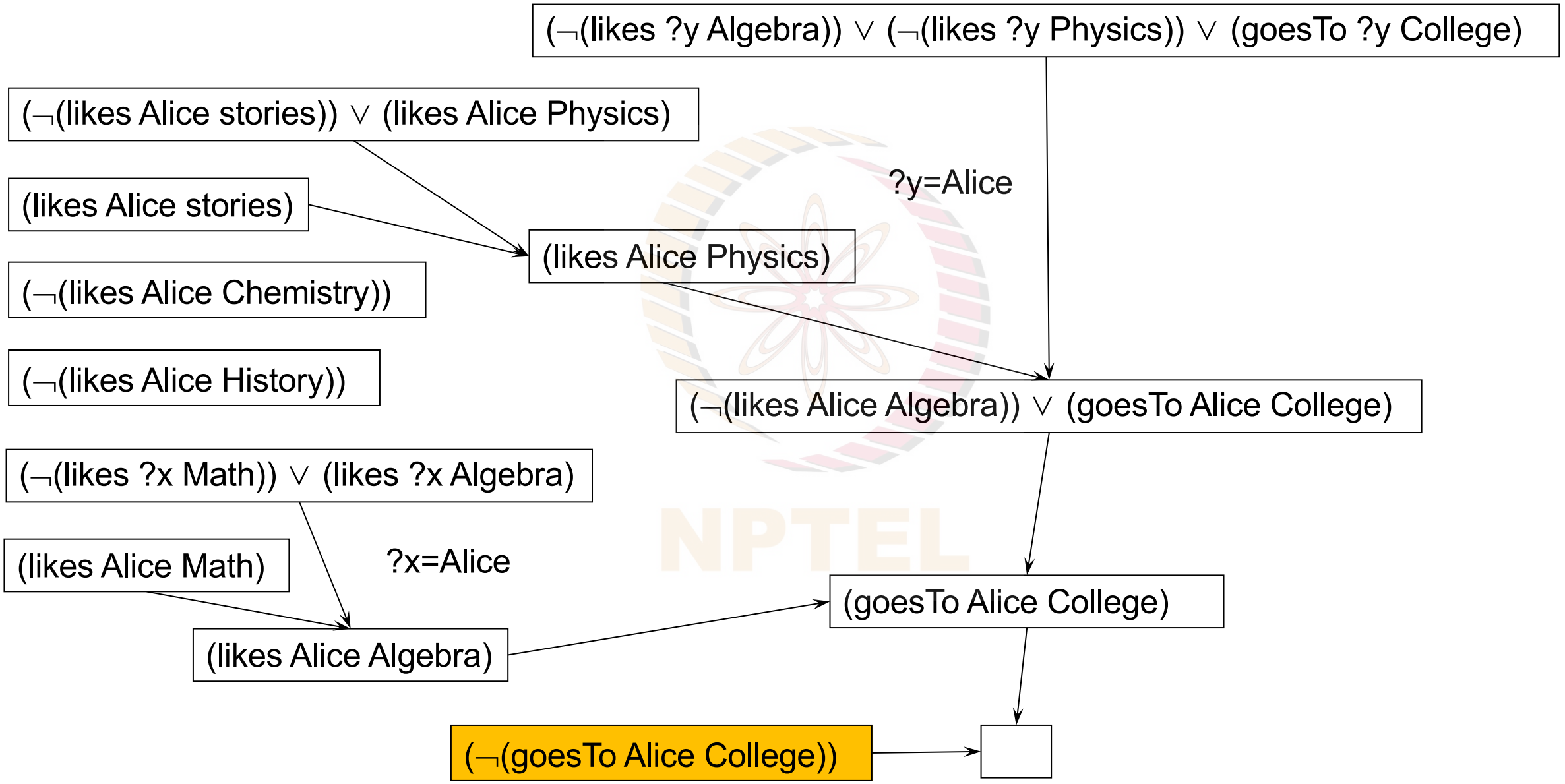
7.(¬(likes Alice History))

8.(¬(goesTo Alice College))

All but the last clause come from the premises.

The last clause is the negated goal clause.

A resolution refutation



Incompleteness of Backward and Forward Chaining

Given the KB,

{(O A B), (O B C), (not (M A)), (M C)}

And the Goal,

(and (O ?x ?y) (not (M ?x)) (M ?y))

Neither Forward Chaining nor Backward Chaining is able to generate a proof.

Both are Incomplete!

Next, we look at a proof method,
the Resolution Refutation System,
that is Sound and Complete for FOL

Recap

A resolution refutation

1.	(O A B))	premise
2.	(O B C))	premise
3.	(¬(M A)))	premise
4.	(M C))	premise
5.	(¬(O ?x ?y)) ∨ (M ?x) ∨ (¬(M ?y))	negated goal

A derivation of the null clause is,

6.	(¬(O ?x C)) ∨ (M ?x)	4,5, ?y=C
7.	(¬(O A ?y)) ∨ (¬(M ?y))	3,5, ?x=A
8.	(¬(M B))	1,7, ?y=B
9.	(M B)	2,6, ?x=B
10.	⊥	8,9

Domain: The Blocks World

1.	(On A B))	premise
2.	(On B C))	premise
3.	(¬(Maroon A)))	premise
4.	(Maroon C))	premise
5.	(¬(On ?x ?y)) ∨ (Maroon ?x) ∨ (¬(Maroon ?y))	negated goal

A derivation of the null clause is,

6.	(¬(On ?x C)) ∨ (Maroon ?x)	4,5, ?y=C
7.	(¬(On A ?y)) ∨ (¬(Maroon ?y))	3,5, ?x=A
8.	(¬(Maroon B))	1,7, ?y=B
9.	(Maroon B)	2,6, ?x=B
10.	⊥	8,9

Domain: People

1.	(LookingAt Jack Anne)	premise
2.	(LookingAt Anne John)	premise
3.	(¬(Married Jack))	premise
4.	(Married John)	premise
5.	(¬(LookingAt ?x ?y)) ∨ (Married ?x) ∨ (¬(Married ?y))	negated goal

A derivation of the null clause is,

6.	(¬(LookingAt ?x John)) ∨ (Married ?x)	4,5, ?y=John
7.	(¬(LookingAt Jack ?y)) ∨ (¬(Married ?y))	3,5, ?x=Jack
8.	(¬(Married Anne))	1,7, ?y=Anne
9.	(Married Anne)	2,6, ?x=Anne
10.	⊥	8,9

Names changed here

Handling Equality

The atomic formula with with equality is $(t_1=t_2)$. This is true iff $t_1^{IA} = t_2^{IA}$.

However even simple problems, where the conclusion is entailed, cannot be proven (without some additional knowledge).

Consider the KB {?N=5, ?M=?N}

Clearly ?M=5 follows, but how does one prove it?

The predicate = is not just any other user defined predicate.

It has well defined semantics and some properties that need to be added to the KB as equality axioms.

Equality Axioms

The following axioms describe some properties of equality that always hold.

- *Reflexivity*: $\forall x\ (x=x)$
- *Symmetry*: $\forall x,y\ (x=y \supset y=x)$
- *Transitivity*: $\forall x,y,z\ ((x=y \wedge y=z) \supset x=z)$
- *Substitution for functions*: for every function f of arity N

  $\forall x_1,y_1, x_2,y_2, \ldots x_N,y_N\ ((x_1=y_1 \wedge x_2=y_3 \wedge \ldots \wedge x_N=y_N) \supset f(x_1,x_2, \ldots, x_N) = f(y_1,y_2, \ldots, y_N)$
- *Substitution for predicates*: for every predicate P of arity N

  $\forall x_1,y_1, x_2,y_2, \ldots x_N,y_N\ ((x_1=y_1 \wedge x_2=y_3 \wedge \ldots \wedge x_N=y_N) \supset P(x_1,x_2, \ldots, x_N) \equiv P(y_1,y_2, \ldots, y_N)$

We convert the above axioms into clause form…..

Equality Axioms in Clause Form

The following axioms describe some properties of equality that always hold.

- *Reflexivity*: $?x=?x$
- *Symmetry*: $\neg ?x=?y \vee ?y=?x$ also written as $?x \neq ?y \vee ?y=?x$
- *Transitivity*: $\neg ?x=?y \vee \neg ?y=?z \vee ?x=?z$ also written as $?x \neq ?y \vee ?y \neq ?z \vee ?x=?z$
- *Substitution for functions*: for every function f of arity

  $?x_1 \neq ?y_1 \vee ?x_2 \neq ?y_3 \vee \ldots \vee ?x_N \neq ?y_N \vee (f(?x_1, ?x_2, \ldots, ?x_N) = f(?y_1, ?y_2, \ldots, ?y_N))$
- *Substitution for predicates*: for every predicate P of arity N

  $?x_1 \neq ?y_1 \vee ?x_2 \neq ?y_3 \vee \ldots \vee ?x_N \neq ?y_N \vee \neg P(?x_1, ?x_2, \ldots, ?x_N) \vee P(?y_1, ?y_2, \ldots, ?y_N))$

Equality Axioms in Clause Form in Prolog style

The following axioms describe some properties of equality that always hold.

- *Reflexivity*: $X=X$
- *Symmetry*: $\neg X=Y \vee Y=X$ also written as $X \neq Y \vee Y=X$
- *Transitivity*: $\neg X=Y \vee \neg Y=Z \vee X=Z$ also written as $X \neq Y \vee Y \neq Z \vee X=Z$
- *Substitution for functions*: for every function f of arity

  $X_1 \neq Y_1 \vee X_2 \neq Y_3 \vee \ldots \vee X_N \neq Y_N \vee (f(X_1, X_2, \ldots, X_N) = f(Y_1, Y_2, \ldots, Y_N))$
- *Substitution for predicates*: for every predicate P of arity N

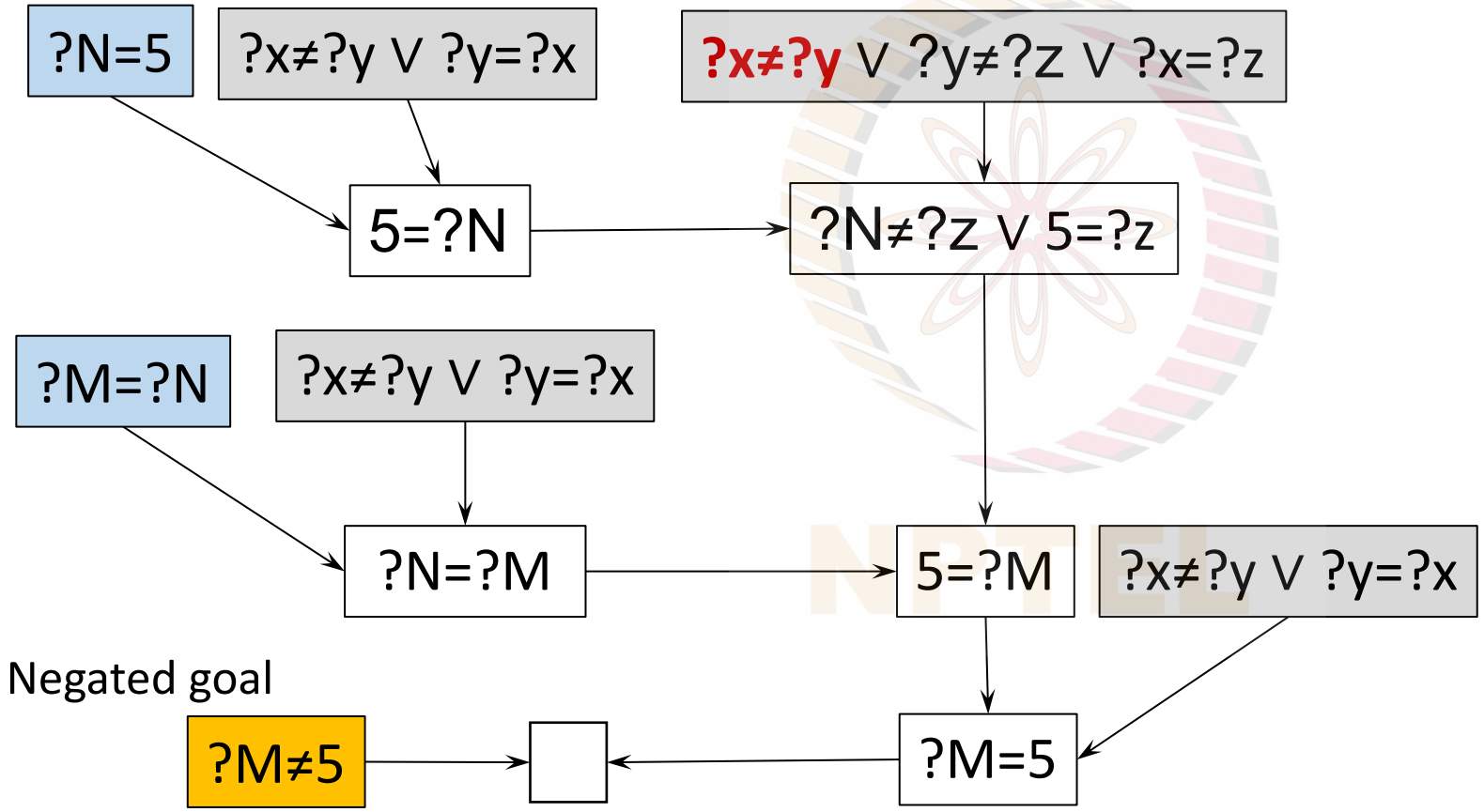
  $X_1 \neq Y_1 \vee X_2 \neq Y_3 \vee \ldots \vee X_N \neq Y_N \vee \neg P(X_1, X_2, \ldots, X_N) \vee P(Y_1, Y_2, \ldots, Y_N))$

In Prolog variables are Capitalized

Yes, M=5

Consider the KB {?N=5, ?M=?N}

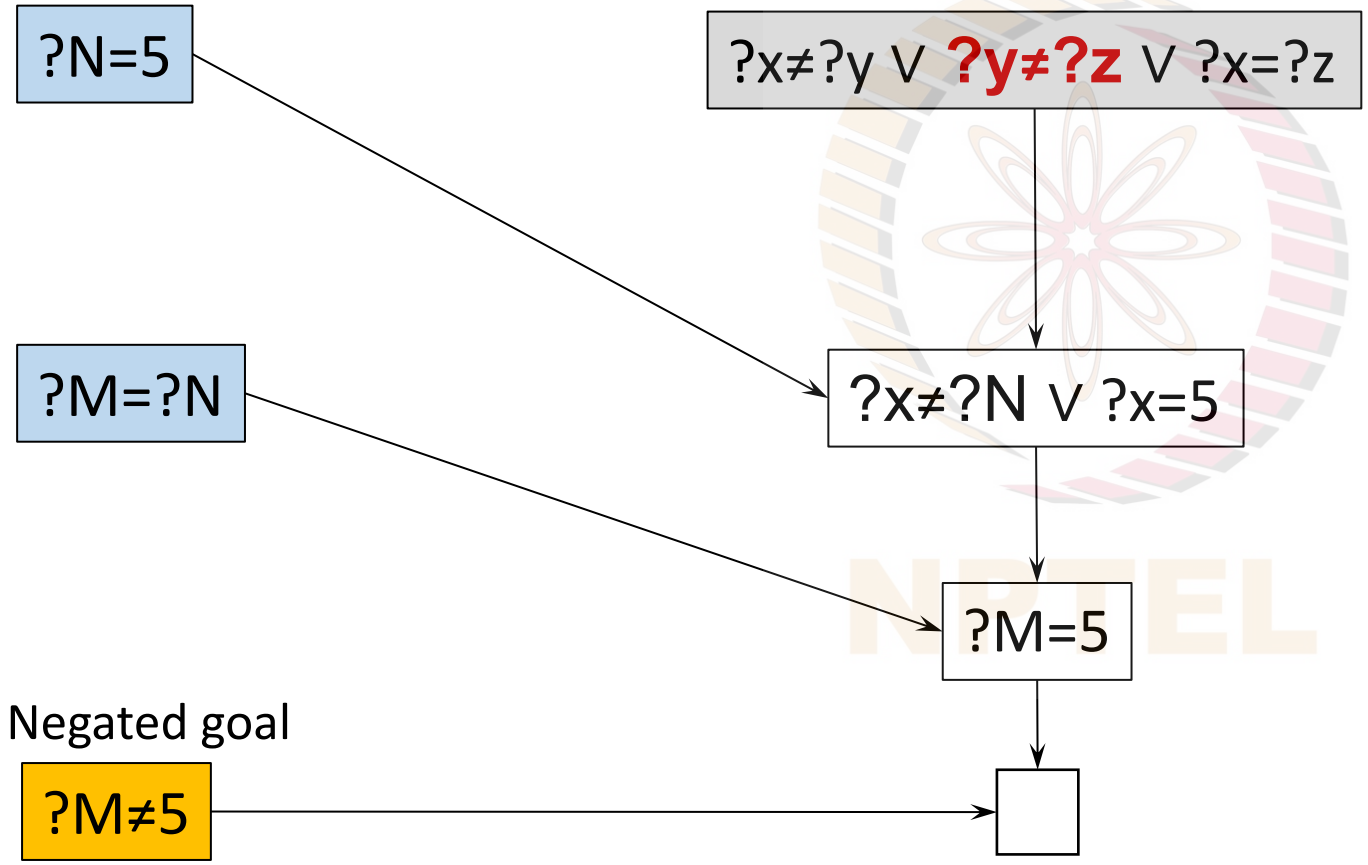
Clearly ?M=5 follows, but how does one prove it?



A much shorter proof!

Consider the KB {?N=5, ?M=?N}

Clearly ?M=5 follows, but how does one prove it?



Who was the surgeon?

A father and his son were crossing the road when they met with an accident. The father died on the spot, and the son was rushed to the hospital, and into the operation theatre. But the surgeon had one look at the boy and said

"I cannot operate on this boy. He is my son".

Question: Who was the surgeon?

Some people find it a trivial question, but there are also some who fail to arrive at the right answer, even after some thought.

For the benefit of the second set, let us solve this using first order logic with equality.

FOL with equality

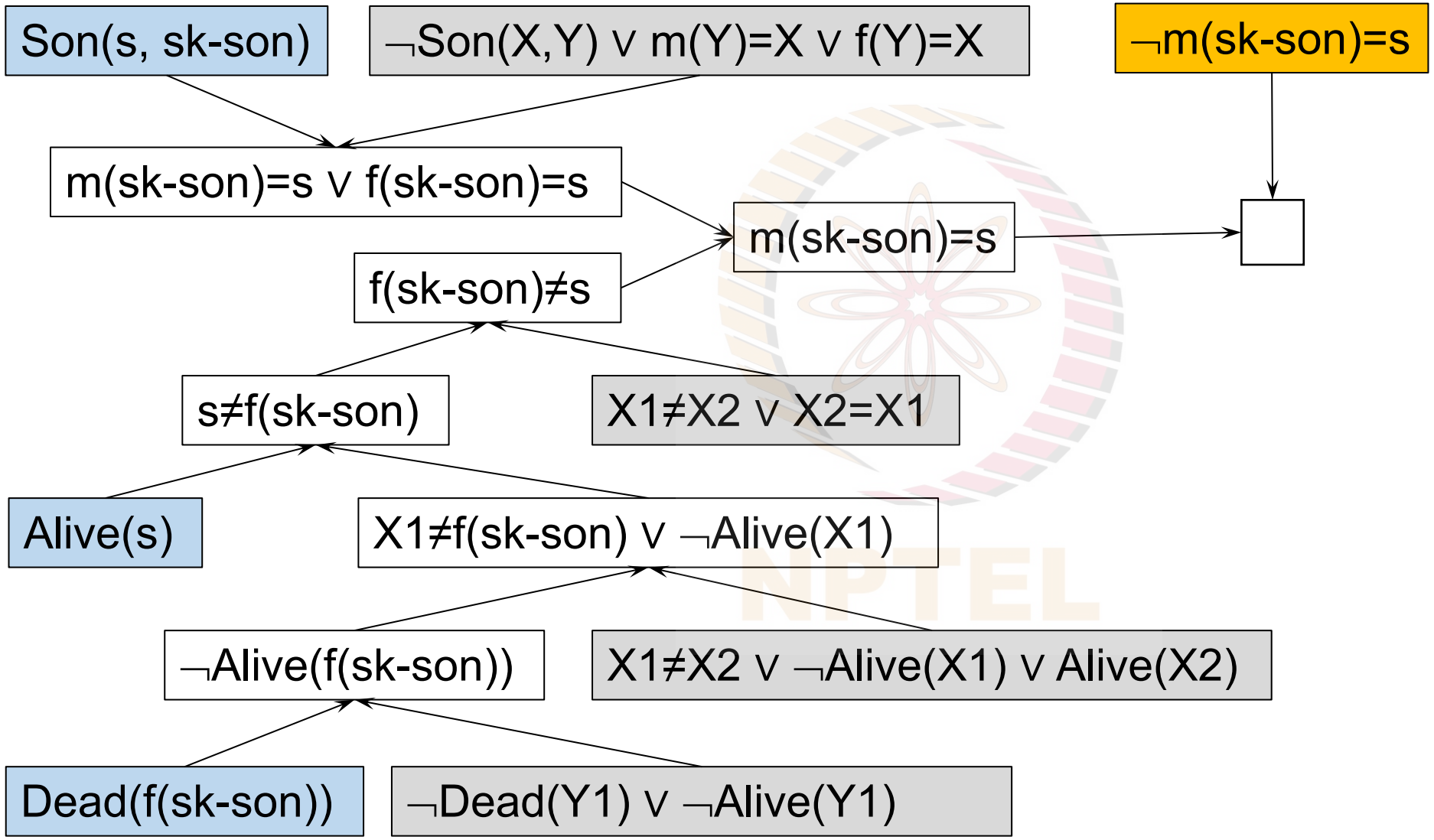
- Let the surgeon be the known individual *s.* Surgeon(s)
- Let Son(X,Y) be read as “The son of X is Y”
- Let m(Y) and f(Y) be two functions read as “mother of Y” and “father of Y”
- You can only be one of father or mother $m(Y)=X \equiv \neg f(Y)=X$
- Then (*could use XOR too)* $Son(X,Y) \equiv (m(Y)=X \vee f(Y)=X)$

  This translates to three clauses $(\neg Son(X,Y) \vee m(Y)=X \vee f(Y)=X)$

  and $(\neg m(Y)=X \vee Son(X.Y))$

  and $(\neg f(Y)=X \vee Son(X.Y))$
- The statement “*He is my son*” $Alive(s) \wedge Son(s, sk\text{-}son)$
- The father died $Dead(f(sk\text{-}son))$
- Dead implies not alive and vice versa $(\neg Dead(Y1) \vee \neg Alive(Y1))$
- Equality (reflexive): $X=X$
- Equality (predicate): $X1 \neq X2 \vee \neg Alive(X1) \vee Alive(X2)$

The mother is the surgeon



The Perils of Inconsistency

- The surgeon story (KB) was consistent.
- A KB is said to be inconsistent if KB $\vdash \alpha$ and KB $\vdash \neg\alpha$
- One can derive *anything* from an inconsistent KB.
- Let us say that the KB derives $(P \wedge \neg P)$ Then,

1.	$P \wedge \neg P$	A given contradiction
2.	P	1, simplification
3.	$\neg P$	1, simplification
4.	$\neg P \vee Q$	3, addition (a random statement)
5.	Q	2, 4, disjunctive syllogism

Gödel's Incompleteness Theorems

In 1931 the German philosopher and mathematician Kurt Gödel published his *Incompleteness Theorem* which dashed the hopes formalizing all mathematical knowledge in *one* formal system.

"*This theorem established that it is impossible to use the axiomatic method to construct a formal system for any branch of mathematics containing arithmetic that will entail all of its truths. In other words, no finite set of axioms can be devised that will produce all possible true mathematical statements, so no mechanical (or computer-like) approach will ever be able to exhaust the depths of mathematics.*

The second incompleteness theorem shows that a formal system containing arithmetic cannot prove its own consistency." https://www.britannica.com/topic/incompleteness-theorem#ref1107613

First incompleteness theorem

"*Any consistent formal system F within which a certain amount of elementary arithmetic can be carried out is incomplete; i.e., there are statements of the language of F which can neither be proved nor disproved in F*."

https://plato.stanford.edu/entries/goedel-incompleteness/

Self reference

- Gödel showed that in any powerful enough axiomatic system there existed statements that were undecidable.
- Such sentences have to do with *self reference*
  - Only powerful enough systems can be self referential
- Hofstadter's book *Gödel, Escher, Bach* is an insightful journey into self reference.
- Bertrand Russell's paradox shows that every set theory that contains an "unrestricted comprehension principle" leads to contradictions.
- The barber paradox, derived from Russell's paradox also has elements of self reference.

If the barber is a person who
shaves everyone who does not shave themselves,
does the barber shave himself?

Some intuition on self-reference

Imagine a machine that derives all true statements.

This example is based on an example given by Raymond Smullyan in which he used “printed” instead of derived, so we had a machine that *printed* all true statements.

Cα	A sentence of the form α-α. The conjugate of α.
D-α	α will be eventually derived if D-α is true
N-α	α cannot be derived if N-α is true
DC-α	α-α will be eventually derived if DC-α is true
NC-α	α-α cannot be derived if NC-α is true

What happens to NC-α when α = NC?

NC-NC says that one cannot derive the conjugate of NC.

But the conjugate of NC is NC-NC.

If NC-NC is true it cannot be derived!

Some other logics

- Second order logic allows one to quantify over predicates as well.

  The first principle of mathematical induction can be stated as

  $\forall P[\{P(base) \wedge \forall x\ (P(x) \supset P(successor(x))\} \supset \forall x P(x)]$

Other logics that are more expressive than FOL

- Modal logics – sentences have the modality of necessity and possibility
  - $\Box\alpha$ read "box α" – the sentence definitely true
  - $\Diamond\alpha$ read as "diamond α" – the sentence is possibly true
- Epistemic and Doxastic logics
  - $K_a\alpha$ - Agent a knows α
  - $B_a\alpha$ - Agent a believes α
- Temporal logics – "α is true at time t"
- Event Calculus – a logic for time, action, and change

FOL is intractable

In First Order Logic we can build logic machines that are

Sound:	If you use valid rules of inference
Complete:	With the Resolution Refutation Method
Consistent:	If the KB is consistent
Semi-decidable:	Halts only if query is true. May loop otherwise.
Intractable:	Haken showed that *proofs can be exponential* in length!

Next we study two tractable subsets of FOL

- Horn Clause Logic – and logic programming
- Description Logics – the logics of noun phrases

Next

Logic Programming
Theorem Proving is Computation